THREATLENS AI

RAG-Assisted Security Analytics, SOC Alert Triage, Attack Correlation and Incident Investigation Platform

Industrial-Level Major Project Baseline and Master Implementation Report

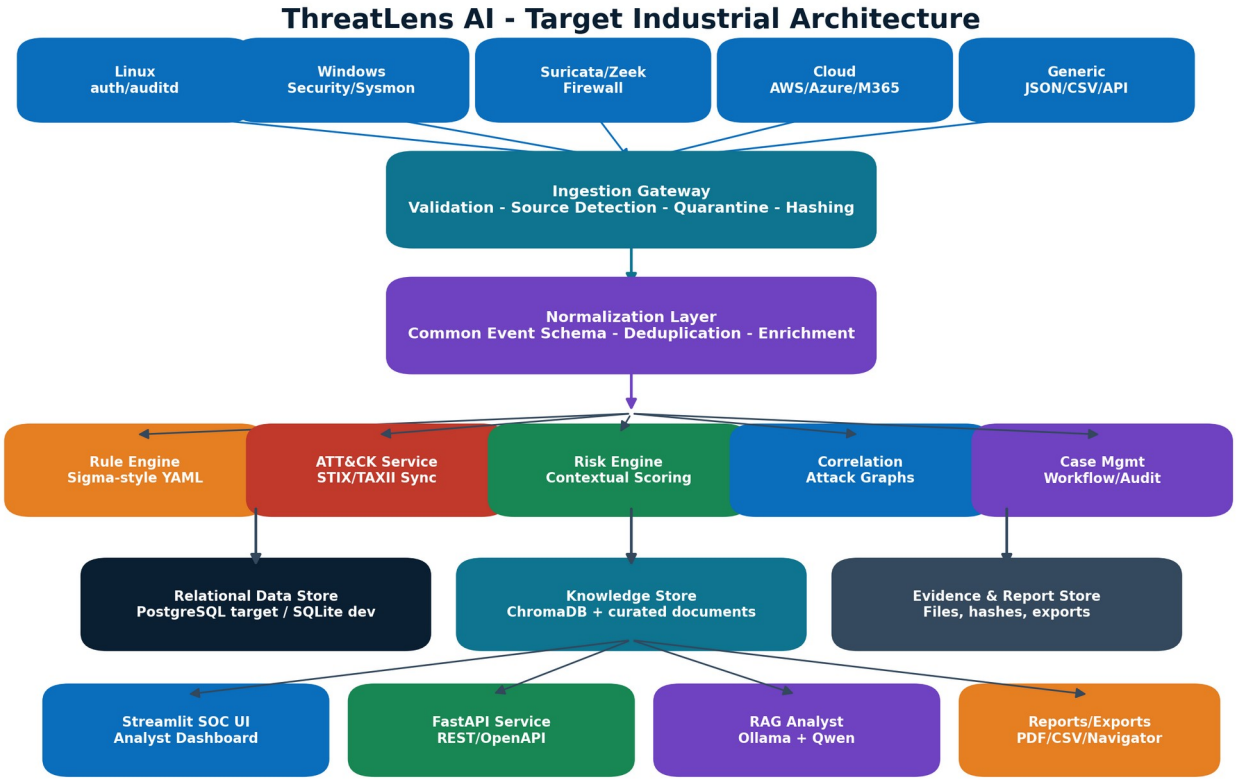


Figure 1. Target industrial architecture of ThreatLens AI

Prepared by	Saketh
Project type	Semester major project - 8 credits
Document purpose	Master baseline for engineering, evaluation, viva and company-level presentation
Current implementation baseline	Core v0.2 complete; persistent SOC schema and Alembic migration complete
Target release	ThreatLens AI v1.0 - industrial demonstration release
Baseline date	18 July 2026

Classification: Academic / Portfolio Engineering Baseline

Document Control and Usage

Field	Value
Document title	ThreatLens AI - Industrial Project Baseline and Master Implementation Report
Version	1.0
Status	Controlled baseline; update after each tagged Git milestone
Owner	Saketh
Audience	Project guide, faculty panel, placement interviewers, company reviewers and development team
Primary use	Scope control, architecture reference, engineering roadmap, evaluation evidence and viva preparation
Review cycle	At every release tag: v0.3, v0.4, v0.5 and v1.0

Important scope statement

ThreatLens AI is being engineered as an advanced security analytics and incident investigation platform. It is not currently a production SIEM replacement, and the report never claims that the prototype can detect every ATT&CK technique from every log source. The target is full ATT&CK catalog awareness, evidence-supported technique mappings, measurable rule coverage and explicit telemetry-gap reporting.

How to Use This Baseline

- Use this report as the source of truth for project scope and architecture. New ideas should be checked against the scope boundaries before implementation.
- Update the Current Implementation Status table after every completed phase and Git tag.
- Use the acceptance criteria to decide whether a feature is complete; a feature is not complete merely because a screen is visible.
- Use the viva question bank to prepare technical explanations, trade-offs and limitations.
- Use the reference architecture to prevent the project from becoming an unstructured collection of scripts or an AI chatbot without cybersecurity depth.

Contents

Sections 1-12	Sections 13-Appendices
1. Executive Summary	2. Project Definition and Vision
3. Problem Statement and Industry Need	4. Scope, Boundaries and Stakeholders
5. Requirements	6. Architecture and Data Flow
7. Ingestion and Normalization	8. Detection Engineering and Rule System
9. MITRE ATT&CK Integration	10. Risk Scoring and Prioritization
11. Correlation, Attack Storylines and Graphs	12. Persistence and Case Management
13. RAG Analyst Assistant	14. Dashboard, API and Reporting
15. Security and Privacy Architecture	16. Database and Deployment Design
17. Testing, Evaluation and Acceptance Criteria	18. Current Implementation Status
19. Master Roadmap and Delivery Plan	20. Limitations, Drawbacks and Risk Register
21. Demonstration Scenarios	22. Viva and Interview Question Bank
23. Conclusion	24. References
Appendices	

1. Executive Summary

ThreatLens AI is an advanced, modular security analytics and Security Operations Center (SOC) investigation platform. It is designed to ingest heterogeneous security telemetry, normalize it into a common event schema, detect suspicious behavior using deterministic and versioned rules, map evidence-supported detections to MITRE ATT&CK, score and correlate alerts into incidents, support case-management workflows, and provide a grounded Retrieval-Augmented Generation (RAG) assistant for analyst explanations and response guidance.

The project addresses a practical SOC problem: security data is distributed across operating systems, endpoint tools, network sensors, cloud platforms and identity providers. Raw events by themselves do not provide an analyst-ready incident story. ThreatLens AI transforms those events through a controlled pipeline: ingestion, validation, normalization, detection, enrichment, scoring, correlation, investigation, reporting and RAG-assisted explanation.

The system deliberately separates deterministic security logic from generative AI. Rules, thresholds, entity relationships, ATT&CK mappings and risk decisions are evidence-driven and testable. The LLM is used only after an incident exists, to retrieve approved knowledge and produce grounded explanations with citations, uncertainty statements and refusal behavior when evidence is insufficient.

The final product is planned as an industrial demonstration architecture rather than a basic student dashboard. The target includes a modular FastAPI service layer, Streamlit analyst interface, PostgreSQL production profile, SQLAlchemy and Alembic persistence, Sigma-inspired YAML rules, MITRE ATT&CK STIX/TAXII synchronization, advanced entity/time correlation, attack graphs, ChromaDB retrieval, a local Ollama-hosted Qwen model, role-based access control, audit logs, automated tests, Docker Compose deployment and professional reports.

1.1 Current verified baseline

- The Linux authentication parser processes the nine-line sample without unparsed records.
- Four prototype detections are operational: repeated failed logins, invalid-user enumeration, successful login after failures and suspicious sudo command execution.
- The prototype produces four alerts and two incidents from the validated sample scenario.
- Unit tests, Ruff code checks and compilation checks pass for the current core.
- A migration-controlled SQLAlchemy schema has been generated and applied with Alembic.
- The database contains tables for analysis sessions, events, alerts, incidents, incident-alert relationships, investigation notes, evidence and audit logs.

Maturity assessment

The current implementation is a tested foundation, not the final industrial release. Core analytics and the persistent schema are complete. Persistence operations, multi-source ingestion, external rule management, full ATT&CK synchronization, advanced correlation, RAG, API security, deployment and final evaluation remain to be implemented.

2. Project Definition and Vision

2.1 Final project title

ThreatLens AI: RAG-Assisted Multi-Source Security Analytics, SOC Alert Triage, Attack Correlation and Incident Investigation Platform.

2.2 Vision statement

To build a trustworthy and explainable SOC investigation platform that converts diverse security telemetry into prioritized, ATT&CK-aligned and auditable incidents, while using retrieval-augmented AI to support analysts without replacing deterministic detection or human judgment.

2.3 Mission

- Reduce time spent manually reading raw logs and grouping duplicate or related alerts.
- Create a common event representation across endpoint, identity, network and cloud telemetry.

- Provide transparent detection logic and evidence for every alert.
- Maintain a synchronized and queryable ATT&CK catalog while avoiding unsupported mappings.
- Create analyst-ready incidents with timelines, entities, confidence, priority and response guidance.
- Preserve investigation history through case management, notes, evidence hashes and audit logs.
- Use RAG to produce grounded explanations from approved knowledge sources.

2.4 Product identity

Dimension	ThreatLens AI position
Product category	Security analytics and incident investigation platform
Primary users	SOC analysts, incident responders, detection engineers, security students and evaluators
Core value	Evidence-to-incident transformation with explainable ATT&CK alignment
AI role	Grounded analyst assistance; never primary attack detection
Deployment model	Local industrial demonstration first; containerized multi-service target
Data model	Persistent, auditable and migration-controlled
Rule philosophy	Versioned, testable, tunable and telemetry-aware

3. Problem Statement and Industry Need

3.1 Core problem

Modern organizations generate security telemetry from authentication systems, operating systems, endpoint sensors, network intrusion detection, DNS, proxies, cloud control planes and identity services. These sources use different schemas and levels of detail. A high volume of isolated events creates alert fatigue, inconsistent prioritization and slow investigations. Analysts must repeatedly answer: What happened? Which entities are affected? Is this one event or a multi-stage attack? Which ATT&CK behavior is supported by evidence? What should be investigated next?

3.2 Specific operational issues solved

Operational issue	Impact	ThreatLens AI response
Heterogeneous logs	Different fields and formats prevent unified analysis	Source-specific parsers convert events to a common schema
Alert overload	Analysts review repeated low-context findings	Deduplication, aggregation and correlation reduce isolated alerts
Inconsistent prioritization	Severity is assigned without business or behavioral context	Contextual risk scoring combines severity, confidence, entity criticality and attack stage
Weak ATT&CK usage	Mappings are manually copied or overclaimed	Catalog synchronization plus evidence-supported rule mappings and coverage gaps
Lost investigation context	Notes and evidence are scattered or disappear after restart	Persistent cases, notes, evidence metadata and audit history
Generic AI output	LLMs may hallucinate or provide unsupported advice	RAG retrieves curated knowledge and requires citations/refusal behavior
Poor project traceability	Student prototypes lack tests, migrations and release discipline	Git branches/tags, automated tests, Alembic migrations and acceptance criteria

3.3 Why the project is relevant for placements

The project demonstrates a combination of cybersecurity knowledge and software engineering: log analysis, detection engineering, ATT&CK, incident response, risk modeling, databases, APIs, testing, RAG, local LLM integration, containerization and secure design. This supports discussion for SOC analyst, security engineer, detection engineer, incident response, cybersecurity analyst, product security and AI-security roles.

4. Scope, Boundaries and Stakeholders

4.1 In-scope capabilities

- Batch and near-real-time ingestion of supported files and API payloads.
- Linux auth/audit telemetry; Windows Security, Sysmon and PowerShell logs; Suricata and Zeek network data; firewall data; selected cloud and identity logs.
- Common event normalization, data-quality reporting, duplicate suppression and quarantine of invalid records.
- Deterministic detection rules with thresholds, time windows, severity, confidence, false-positive notes, ATT&CK mappings and playbook references.
- Risk scoring, alert deduplication, entity correlation, temporal sequences and attack-storyline generation.
- Persistent analysis history, incident workflow, assignment, notes, evidence and audit logs.
- Full Enterprise ATT&CK catalog awareness through official data synchronization; observed coverage is limited to supported telemetry and rules.
- Grounded RAG assistant, report generation, CSV export and ATT&CK Navigator-compatible layer export.
- Automated tests, code quality, migrations, Docker Compose and reproducible demonstration scenarios.

4.2 Explicitly out of scope for version 1.0

- Replacement of an enterprise SIEM, EDR, SOAR or threat-intelligence platform.
- Unrestricted active response against real production systems.
- Packet capture at carrier-scale throughput or petabyte-scale retention.
- Claiming detection of every ATT&CK technique without the required telemetry.
- Autonomous LLM decisions that block users, isolate hosts or close incidents without analyst approval.
- Production multi-tenant SaaS billing, global high availability or regulatory certification.

4.3 Stakeholders and roles

Role	Responsibilities in the system
SOC Analyst	Review incident queue, timelines, entities, ATT&CK evidence, notes and recommended next steps
Incident Responder	Containment tracking, evidence registration, case resolution and post-incident reporting
Detection Engineer	Create/version/test rules, map ATT&CK, tune false positives and evaluate coverage
Administrator	Manage users, connectors, configurations, retention and health
Auditor/Guide	Review immutable actions, test evidence, architecture and completion status
Viewer/Manager	Read executive metrics and reports without modifying cases

5. Requirements

5.1 Functional requirements

ID	Requirement
FR-01	Accept one or more supported log files and reject unsupported or unsafe inputs.
FR-02	Detect log source automatically or allow an explicit source override.
FR-03	Normalize source fields into a versioned common event schema.
FR-04	Persist analysis sessions, events, alerts, incidents and relationships transactionally.
FR-05	Load detection rules from external YAML with metadata, version, conditions and tests.
FR-06	Evaluate threshold, aggregation, temporal and ordered correlation logic.
FR-07	Synchronize official Enterprise ATT&CK catalog data and maintain version metadata.
FR-08	Map rules and incidents only when evidence supports the mapping.
FR-09	Calculate risk, confidence, priority and reason codes.
FR-10	Create cases, assign analysts, update status, add notes and register

ID	Requirement
	evidence.
FR-11	Provide history and metrics after application restart.
FR-12	Generate technical and executive reports and structured exports.
FR-13	Retrieve knowledge and produce cited RAG explanations for an incident.
FR-14	Record audit events for security-sensitive operations.
FR-15	Expose controlled API endpoints for ingestion, cases, reports and health.

5.2 Non-functional requirements

ID	Requirement
NFR-01 Reliability	A failed persistence operation rolls back the complete analysis transaction.
NFR-02 Explainability	Every alert includes rule ID, evidence, rationale, mapping and risk reason.
NFR-03 Security	Input validation, least privilege, secret management, RBAC and audit logging.
NFR-04 Maintainability	Modular services, typed models, migrations, versioned schemas and rule tests.
NFR-05 Performance	Defined ingestion and query benchmarks with measurable latency and throughput.
NFR-06 Portability	Development on SQLite; target PostgreSQL without rewriting domain logic.
NFR-07 Reproducibility	Docker Compose, pinned dependencies, sample datasets and scripted setup.
NFR-08 Observability	Structured application logs, health checks and processing metrics.
NFR-09 Privacy	Minimize, redact and retain security data according to configuration.
NFR-10 AI safety	Source citations, context limits, prompt-injection defenses and refusal behavior.

6. Architecture and Data Flow

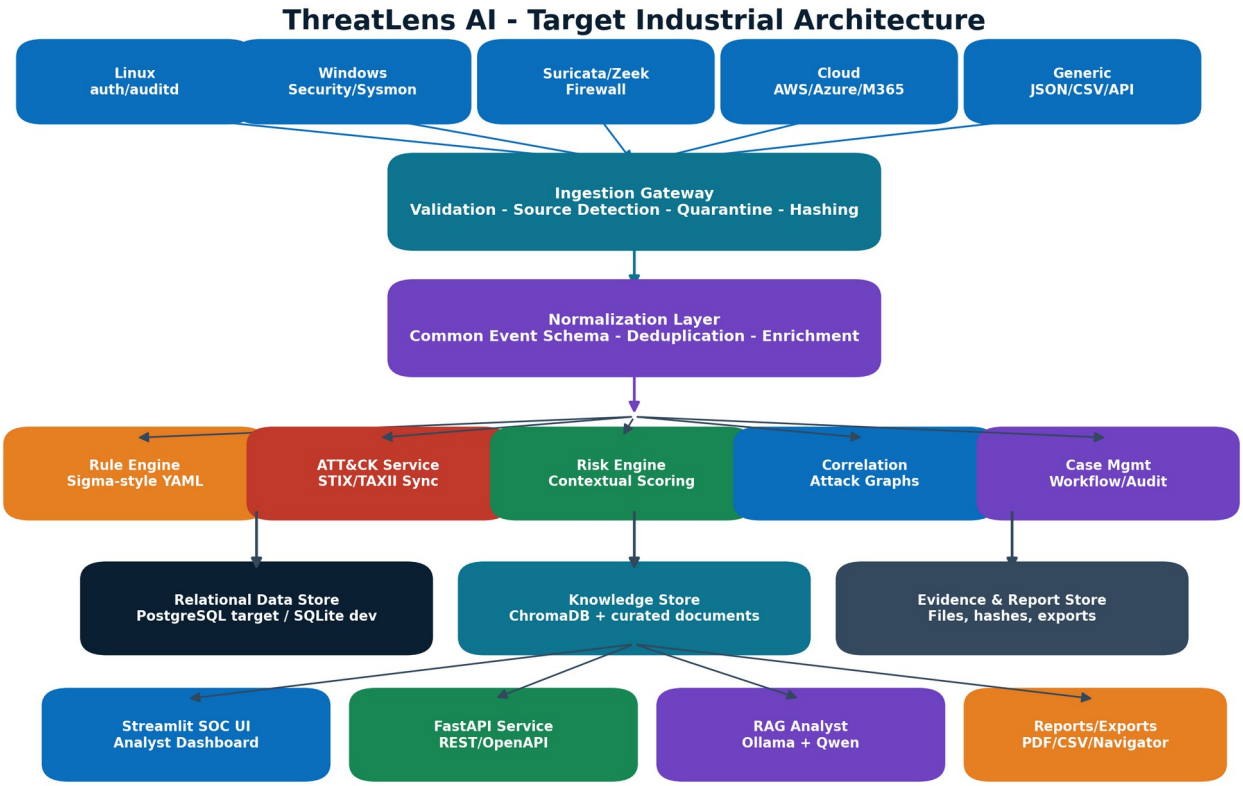


Figure 2. Layered target architecture

6.1 Architectural style

The recommended final architecture is a modular monolith with explicit service boundaries. This keeps the project feasible for a semester while preserving industrial engineering discipline. The user interface, API, ingestion, detection, ATT&CK synchronization, correlation, persistence, reporting and RAG modules are separated in code and can later be deployed as independent services if required.

6.2 Layer responsibilities

Layer	Responsibilities
Presentation	Streamlit SOC dashboard, case views, charts, forms and report downloads
API	FastAPI endpoints, validation, authentication, authorization and OpenAPI documentation
Application services	Analysis orchestration, case workflow, reporting, ATT&CK sync and RAG orchestration
Domain	Event schema, alerts, incidents, risk policy, rule evaluation and correlation models
Infrastructure	SQLAlchemy repositories, Alembic, Chroma, Ollama, file storage and connectors
Cross-cutting	Configuration, secrets, structured logging, metrics, security, audit and tests

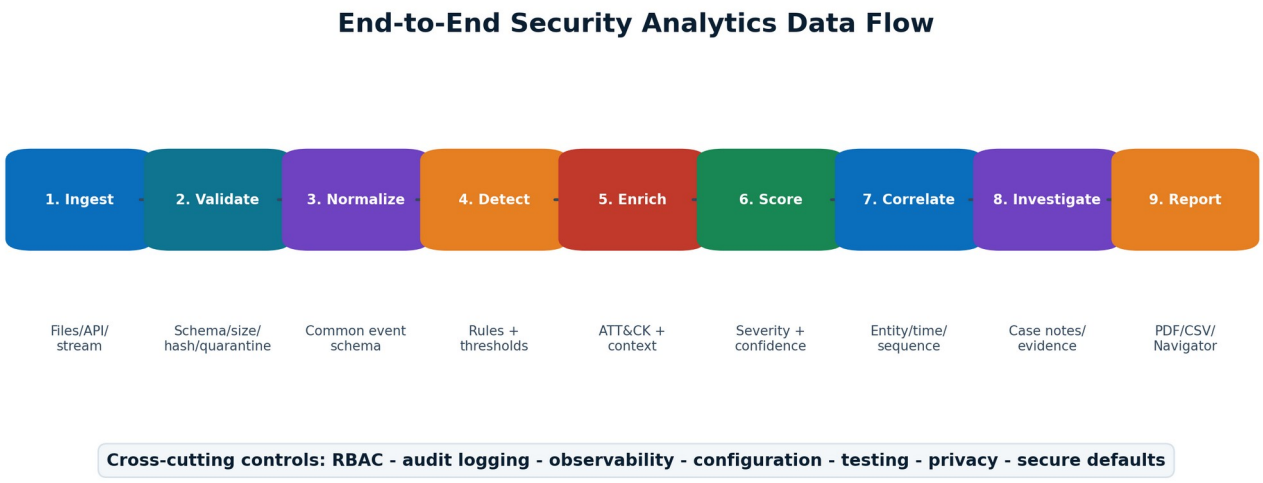


Figure 3. End-to-end data flow

6.3 Why a modular monolith instead of microservices now

- A microservice architecture would add network, deployment, observability and distributed-transaction complexity before the security logic is mature.
- A modular monolith allows clear boundaries and testability while keeping debugging and deployment manageable.
- Docker Compose can still deploy separate UI, API, database, vector store and LLM services for the final demonstration.
- The architecture preserves future extraction points for high-volume workers or specialized connectors.

7. Ingestion and Normalization

7.1 Planned connectors

Category	Initial sources	Future expansion
Linux	auth.log, auditd, journald export	sudo, process, file and network audit rules
Windows	Security CSV/JSON, Sysmon, PowerShell	Defender and event-channel collectors

Category	Initial sources	Future expansion
	Operational	
Network	Suricata eve.json, Zeek conn/dns/http, firewall CSV	Proxy, NetFlow and PCAP-derived metadata
Cloud/Identity	AWS CloudTrail, Azure/Entra sign-in, Microsoft 365 audit	GCP audit and SaaS identity connectors
Generic	JSON, CSV and controlled API payload	Connector SDK and mapping templates

7.2 Ingestion controls

- File extension, MIME type, size and schema validation.
- SHA-256 hashing for duplicate-ingestion detection and evidence integrity.
- Safe decoding using UTF-8 with BOM handling and controlled error replacement.
- Per-source parser version and normalization-schema version.
- Quarantine collection for malformed records with error reasons.
- Idempotency and fingerprints to avoid duplicate events within one analysis.
- Parser metrics: total, parsed, dropped, quarantined and unsupported records.

7.3 Common event schema

Field group	Representative fields
Identity	event_uuid, external_event_id, analysis_session_id, source_type, schema_version
Time	timestamp, ingestion_time, timezone, sequence number
Entities	source_ip, destination_ip, hostname, username, process_name, file_path, domain
Behavior	event_type, event_action, event_outcome, command_line, protocol, port
Evidence	raw_message, normalized_message, attributes_json, event_fingerprint
Quality	parse_status, parser_version, validation_errors, confidence

8. Detection Engineering and Rule System

8.1 Detection philosophy

ThreatLens AI will not remain dependent on hardcoded if-statements. The target rule system is inspired by Sigma: YAML metadata, log-source requirements, selections, conditions, severity, false-positive guidance, ATT&CK tags and response references. Sigma rules are YAML files intended to describe suspicious behavior in log data, and Sigma correlation rules support event counts, temporal relationships and grouping. [2][3]

8.2 Rule lifecycle

Stage	Required controls
Draft	Unique UUID, owner, description, data source and expected behavior
Test	Positive fixtures, negative fixtures, boundary tests and false-positive examples
Review	ATT&CK validation, peer review, severity and confidence review
Release	Version, changelog, status and compatibility metadata
Tune	Filters, allowlists, thresholds and environment-specific overrides
Retire	Deprecation reason, replacement rule and migration notes

8.3 Target detection catalog

The version 1.0 target is 35-45 tested detections across authentication, execution, persistence, privilege escalation, defense impairment, discovery, lateral movement, command and control, exfiltration indicators and network reconnaissance. Quality is prioritized over an inflated rule count.

Domain	Example rules
Authentication	Password guessing, password spraying, invalid-account enumeration, success-after-failure, impossible or unusual source
Linux privilege	Sensitive sudo commands, account/group modification, cron persistence, SSH key changes
Windows execution	Encoded PowerShell, suspicious command shell, LOLBin behavior, credential dumping indicators
Defense controls	Security service stopped, logging disabled, policy modified
Network	Port scanning, DNS anomalies, C2 indicators, suspicious outbound connections, excessive failed connections
Cloud/Identity	Risky sign-in, role/policy changes, access-key creation, abnormal API operations

8.4 Example industrial rule record

```
title: Suspicious Sudo Access to Sensitive File
id: 9be5f4e1-fd90-4a6b-b6eb-4efa3d572b94
status: test
version: 1
logsource:
  product: linux
  service: auth
detection:
  selection:
    event_type: sudo_command
    command_line|contains:
      - /etc/shadow
      - /etc/sudoers
  condition: selection
severity: high
confidence: 0.85
attack:
  primary: T1548.003
falsepositives:
  - Approved administrative maintenance
response:
  playbook: linux_privilege_abuse
```

9. MITRE ATT&CK Integration

9.1 Full catalog awareness versus detection coverage

Critical technical distinction

ThreatLens AI will synchronize the full Enterprise ATT&CK catalog, but it will not claim that every technique is detectable from every telemetry source. The system maps an alert only when a validated rule and event evidence support the mapping. Missing telemetry is displayed as a detection gap, not hidden.

The current ATT&CK v19 release lists 15 Enterprise tactics, 222 techniques and 475 sub-techniques. The v19 release also introduced the Stealth and Defense Impairment tactics by splitting the earlier Defense Evasion concept. [1] The project must store ATT&CK version metadata because the framework changes over time.

9.2 Data synchronization

- Use the official MITRE STIX/TAXII data source or versioned ATT&CK dataset. MITRE provides API access to the STIX knowledge base through TAXII. [4]
- Synchronize tactics, techniques, sub-techniques, mitigations, detection strategies, data components and relationships.
- Preserve ATT&CK object IDs, versions, modified timestamps, revoked/deprecated flags and source version.
- Create a mapping table between ThreatLens rule versions and one or more ATT&CK objects.
- Generate ATT&CK Navigator-compatible JSON layers for observed and covered techniques.

9.3 Mapping policy

Policy	Implementation requirement
Primary mapping	The best-supported technique or sub-technique for the rule behavior
Secondary mapping	Additional behavior only when evidence and rule intent justify it
Justification	Human-readable explanation of why the rule maps to the technique
Confidence	Numeric confidence based on telemetry quality and rule specificity
Telemetry requirement	Fields and source types necessary for the mapping
Gap status	Covered, partial, unsupported telemetry, no rule, deprecated or observed

9.4 Current mapping corrections

- The prototype maps brute-force behavior to T1110; the final rule set will distinguish password guessing, spraying, cracking and credential stuffing when event patterns permit.
- Suspicious sudo behavior should use T1548.003 (Sudo and Sudo Caching) as the primary mapping rather than only T1078 Valid Accounts.
- Mappings will move from a small Python dictionary to versioned database records synchronized with ATT&CK data.

10. Risk Scoring and Prioritization

10.1 Purpose

Severity alone does not represent incident priority. ThreatLens AI will compute a transparent score using the rule severity, detection confidence, asset criticality, identity privilege, external exposure, attack-stage progression, event volume, related-alert count, threat-intelligence context and data quality.

10.2 Proposed score model

Factor	Example weight	Reason
Base severity	0-35	Rule impact and expected consequence
Detection confidence	0-15	Specificity and telemetry quality
Asset criticality	0-15	Business importance of affected host/service
Identity privilege	0-10	Administrative or sensitive-account involvement
Attack progression	0-10	Multiple ATT&CK stages or success indicator
Exposure/context	0-10	Internet source, threat intelligence or public service
Volume and recurrence	0-5	Frequency, burst and repeated incidents

The final score is capped at 100 and accompanied by reason codes. Priority is derived from score and business policy: P1 critical, P2 high, P3 medium and P4 low. A score must be reproducible; the RAG assistant cannot alter it.

10.3 Required safeguards

- Store every factor and contribution, not only the final score.
- Version the scoring policy and preserve the version used for old incidents.
- Provide organization-level configuration for asset criticality and allowlists.
- Evaluate calibration using labeled test scenarios and false-positive review.

11. Correlation, Attack Storylines and Graphs

11.1 Current versus target correlation

The current prototype groups alerts mainly by source IP, which is useful for a first end-to-end pipeline but insufficient for an industrial investigation. The target correlation engine uses multiple entities, time windows, ordered sequences, ATT&CK stages and relationship confidence.

11.2 Correlation methods

Method	Example
Aggregation	At least ten failed logins from one source within five minutes
Entity correlation	Same username, hostname, process, domain or cloud principal
Temporal sequence	Failures followed by success, privilege change and outbound connection
Cross-source correlation	Windows login plus Sysmon process plus Suricata network alert
Graph correlation	Nodes for users, hosts, IPs and alerts; weighted relationship edges
Suppression	Merge duplicate alerts or recurring noisy events into one finding

11.3 Incident storyline example

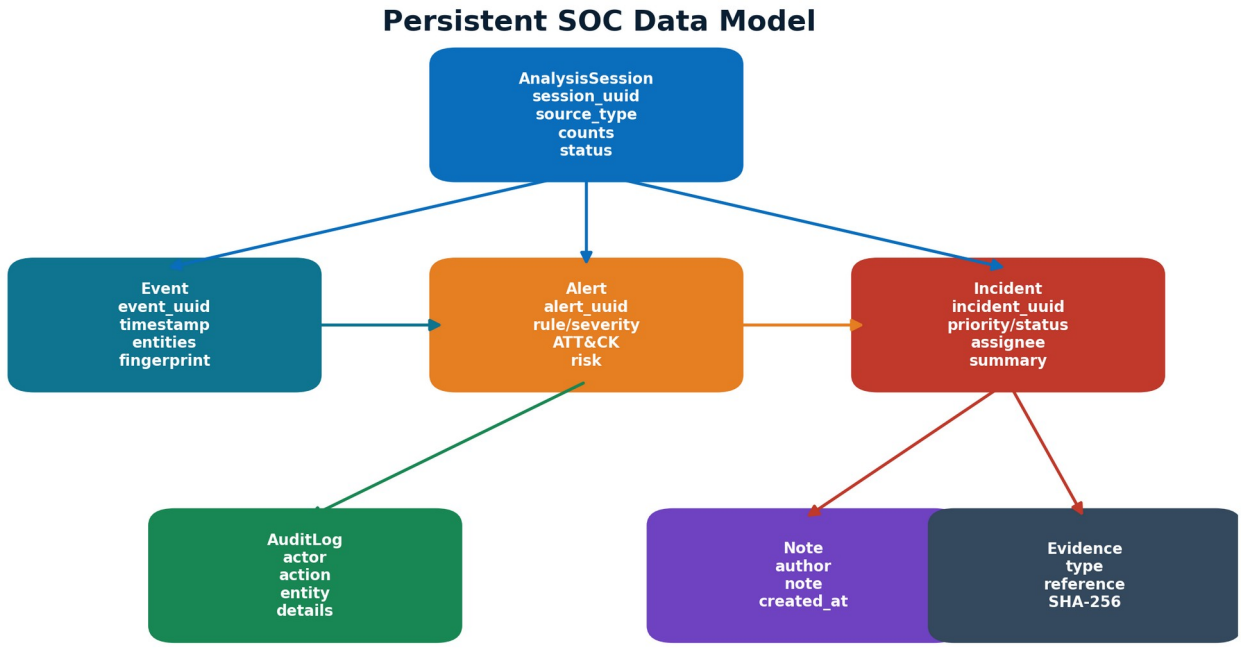
Example

Password guessing from an external IP -> successful root login -> sensitive sudo command -> suspicious DNS/HTTPS connection. The incident is one investigation story, not four disconnected alerts.

11.4 Attack graph output

- Entity nodes: user, host, IP, process, file, domain and cloud resource.
- Evidence nodes: events, alerts and ATT&CK techniques.
- Edges: authenticates-to, executes-on, connects-to, modifies, related-by-time and supports-technique.
- Outputs: interactive graph, machine-readable JSON and graph summary in the incident report.

12. Persistence and Case Management



Key design properties: UUID identifiers - foreign keys - cascade rules - fingerprints - indexes - migration control - auditability

Figure 4. Persistent SOC data model

12.1 Current database status

SQLAlchemy models and an Alembic migration have been created and applied. The current schema includes analysis_sessions, events, alerts, incidents, incident_alerts, investigation_notes, evidence, audit_logs and alembic_version. This is completed. The application service that writes and retrieves real analyses is the next implementation step.

12.2 Transactional persistence

One analysis must be stored atomically. If an event, alert, incident or relationship fails, the transaction rolls back so the database does not contain a partial analysis. SQLAlchemy sessions provide explicit commit and rollback behavior, while Alembic maintains version-controlled schema changes. [8][9]

12.3 Case workflow

State	Meaning	Allowed next states
Open	Newly correlated incident awaiting review	Investigating, False Positive
Investigating	Analyst is validating scope and evidence	Contained, Resolved, False Positive
Contained	Immediate threat control completed	Investigating, Resolved
Resolved	Investigation and remediation complete	Reopened/Investigating by controlled action
False Positive	Finding is benign and documented	Reopened/Investigating

12.4 Evidence and auditability

- Evidence records store a reference, description, type and optional SHA-256 integrity hash.
- Analyst notes are timestamped and associated with an incident.
- Audit logs record actor, action, entity, previous state and new state.
- Reports include the incident UUID, analysis UUID, rule versions and ATT&CK version.

13. RAG Analyst Assistant

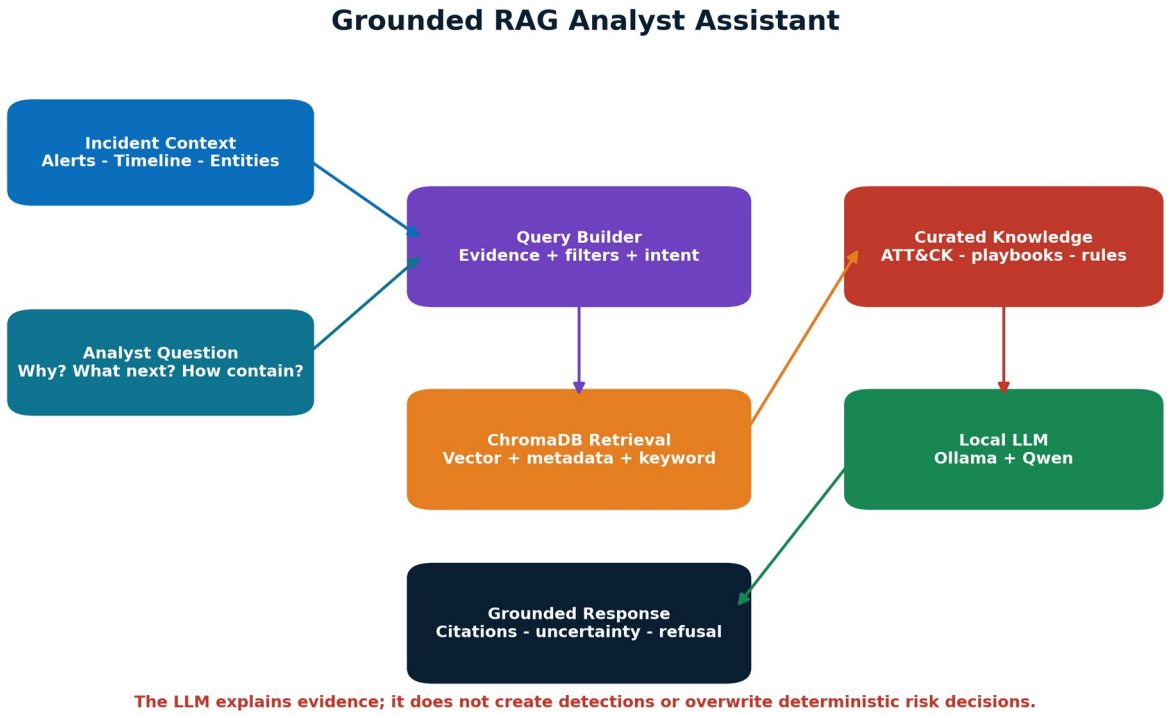


Figure 5. Grounded RAG assistant architecture

13.1 Why RAG is included

A normal LLM can produce generic or incorrect security advice. RAG retrieves relevant documents before generation. Chroma supports collections of documents, metadata, embeddings and similarity queries, as well as metadata and document filters. [5][6] ThreatLens AI will use this for MITRE explanations, internal detection documentation, SOC playbooks and approved response guidance.

13.2 Planned knowledge base

- ATT&CK techniques, sub-techniques, detection strategies and mitigations.
- ThreatLens rule documentation, telemetry requirements and false-positive notes.
- Incident response playbooks for authentication, privilege, endpoint, network and cloud incidents.
- Log-source field documentation and parser behavior.
- Company or project policies, escalation matrix and report templates.

13.3 LLM runtime and model

The local runtime target is Ollama, which exposes a local API for programmatic model interaction. [7] The initial model candidate is Qwen3 4B for lower resource usage, with Qwen3 8B as an optional upgrade when hardware permits. The final model choice will be benchmarked for groundedness, latency, memory use and instruction-following rather than selected only by model size.

13.4 Guardrails

Control	Expected behavior
Context-only answer	Use retrieved evidence and incident facts; do not invent missing details
Citations	Return document IDs, titles and relevant chunks
Insufficient evidence	State that the available context is insufficient
Prompt injection defense	Treat retrieved text as data, apply source allowlists and remove unsafe instructions
No autonomous action	Never block, isolate, delete or close a case without explicit workflow authorization
Output validation	Structured response schema, length limits and prohibited claims

13.5 RAG evaluation

- Retrieval precision and recall on a labeled question set.
- Groundedness: percentage of claims supported by retrieved sources.
- Citation correctness and completeness.
- Refusal accuracy when the knowledge base lacks the answer.
- Latency and memory use on the chosen laptop.

14. Dashboard, API and Reporting

14.1 Analyst interface

- Overview metrics: events, alerts, incidents, priority, open cases and trend.
- Ingestion: multi-file upload, source detection, validation and parser statistics.
- Event explorer: filters, search, entity pivots and raw/normalized comparison.
- Alert view: evidence, rule metadata, risk breakdown and ATT&CK mapping.
- Incident queue: status, priority, assignee, SLA and confidence.
- Case detail: storyline, graph, timeline, notes, evidence and audit history.
- ATT&CK coverage: full matrix, observed techniques, covered rules and gaps.
- RAG assistant: incident-grounded questions and cited answers.
- Reports and exports: technical PDF, executive PDF, CSV and Navigator JSON.

14.2 API target

FastAPI is planned for the backend because it is based on OpenAPI and provides automatic interactive API documentation. [10] The UI will call the service layer instead of directly accessing the database in the final architecture.

Endpoint group	Examples
Health	GET /health, GET /ready, GET /version
Ingestion	POST /analyses, GET /analyses/{uuid}, GET /analyses
Events/alerts	GET /events, GET /alerts, GET /alerts/{uuid}
Incidents	GET /incidents, PATCH /incidents/{uuid}, POST /incidents/{uuid}/notes
Evidence	POST /incidents/{uuid}/evidence, GET evidence metadata
ATT&CK	POST /attack/sync, GET /attack/coverage, GET /attack/techniques/{id}
RAG	POST /assistant/query with incident UUID and question
Reports	POST /reports, GET /reports/{uuid}

14.3 Report types

Report	Audience	Contents
Technical incident report	SOC/IR	Evidence, alerts, timeline, ATT&CK, entities, risk, notes, response actions
Executive summary	Management	Business impact, priority, scope, status and next decisions
Detection coverage report	Detection engineering	Rules by source and ATT&CK coverage/gaps
Analysis quality report	Engineering/guide	Parser quality, test results, performance and known limitations

15. Security and Privacy Architecture

15.1 Security controls

Area	Controls
Authentication	Local accounts or identity integration; password hashing; session expiration
Authorization	RBAC roles: administrator, detection engineer, analyst, viewer
Input security	File size/type checks, path safety, schema validation, decompression limits
Secrets	Environment variables or secret store; never commit keys or credentials
Database	Least-privilege account, parameterized ORM queries, migrations and backups
API	Validation, rate limits, CORS policy, request IDs and authentication
Audit	Immutable logical audit trail for case changes and administrative operations
RAG	Approved-source allowlist, prompt-injection filtering, output validation and citations
Reports	Sensitive-field redaction, authorization and controlled storage

15.2 Privacy considerations

- Security logs can contain usernames, IP addresses, commands, file paths and authentication details.
- Datasets used for demonstration should be synthetic or authorized and should not expose real credentials.
- Retention and deletion must be configurable; unnecessary raw content should not be stored indefinitely.
- Reports should support redaction and role-based access.
- The local LLM approach reduces external data transmission, but local files and model prompts still require access controls.

16. Database and Deployment Design

16.1 Database strategy

Profile	Technology	Purpose
Development	SQLite + WAL	Fast local setup and deterministic tests
Industrial demonstration	PostgreSQL	Concurrency, indexing, JSON querying and operational realism
RAG	ChromaDB	Embedding and document retrieval with metadata
Evidence	Controlled file/object store	Reports, source files and exported artifacts

16.2 Migration discipline

- Every schema change creates an Alembic migration.
- Migrations are reviewed and applied in sequence; the database is never edited manually as the primary method.
- Integration tests create isolated temporary databases.
- Production backup and restore procedures are tested before the final demonstration.

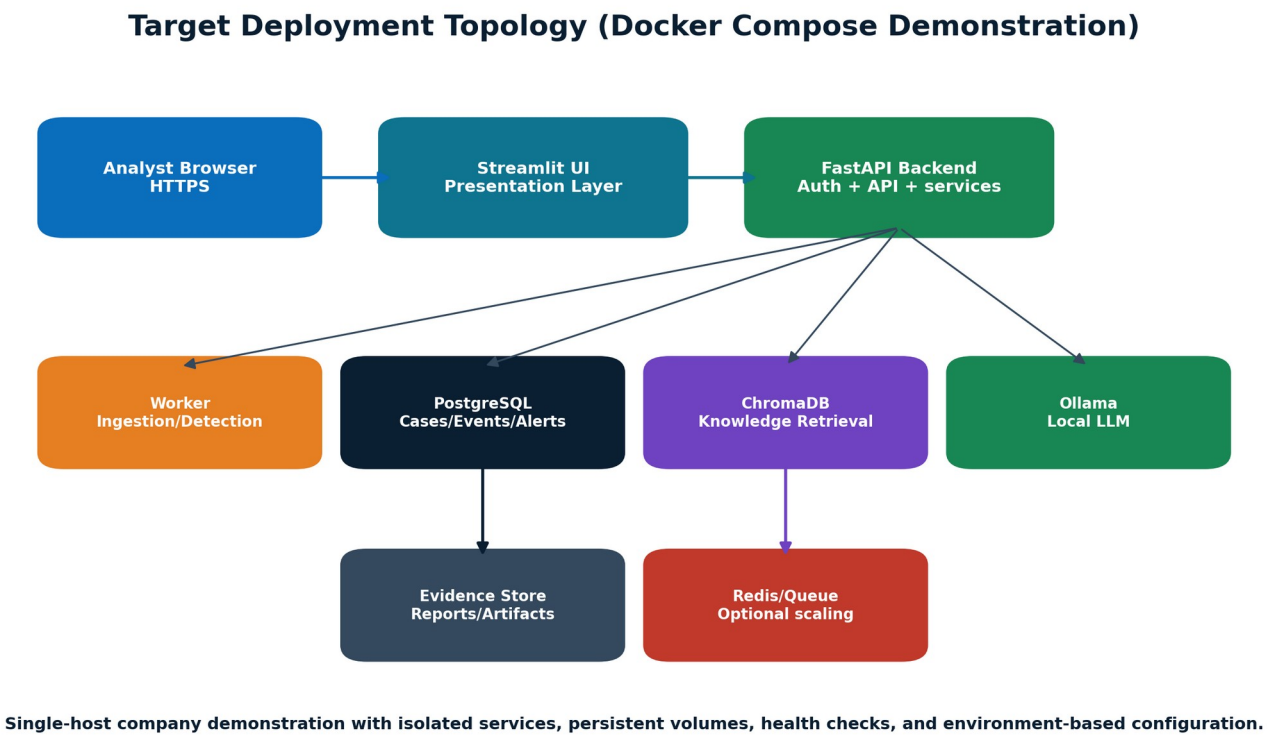


Figure 6. Target Docker Compose deployment topology

Docker Compose is planned because it defines and runs a multi-container application through a single YAML configuration, including services, networks and volumes. [11] The final demonstration can start the UI, API, database, vector store, LLM and optional worker consistently.

17. Testing, Evaluation and Acceptance Criteria

17.1 Test pyramid

Test level	Scope
Unit	Parsers, rule operators, risk factors, mappings and helpers
Integration	Database transactions, migrations, repositories, Chroma and Ollama clients
Pipeline	Log -> events -> alerts -> incidents -> report
API	Validation, permissions, error handling and OpenAPI contracts
UI	Critical workflows and state persistence

Test level	Scope
Performance	Ingestion throughput, query latency, report generation and RAG latency
Security	Input abuse, authorization, prompt injection, secrets and dependency review
User acceptance	Demonstration scenarios and evaluator checklist

17.2 Quantitative targets

Metric	Version 1.0 target
Core automated test coverage	At least 80%; critical rule/correlation paths at least 90%
Parser quality on supported fixtures	At least 98% records parsed; malformed records explicitly quarantined
Detection fixture accuracy	All positive fixtures detected; all defined negative fixtures remain quiet
Duplicate ingestion	Same source and engine version rejected or explicitly overridden
Database atomicity	No partial analysis after simulated failure
Dashboard restart persistence	History and cases remain after restart
RAG groundedness	Target at least 90% supported claims on evaluation set
RAG citation correctness	Target at least 95% correct cited source IDs
Industrial demo startup	One documented Docker Compose command

17.3 Definition of done

- Code implemented and reviewed.
- Positive, negative and boundary tests pass.
- Ruff and type checks pass.
- Database migrations are synchronized.
- Feature is visible in the UI/API and persists correctly.
- Documentation, screenshots and acceptance evidence are updated.
- Git commit and release milestone are recorded.

18. Current Implementation Status

Status is reported honestly as of 18 July 2026. COMPLETED means validated with working output or applied migration. PARTIAL means a prototype exists but the industrial target is not yet complete.

Workstream	Current status	Evidence completed	Industrial target
Project repository, Python 3.11 environment and dependency setup	**COMPLETED**	Virtual environment, requirements, editable package and Git repository	Pinned dependencies, CI workflow and Docker image
Linux auth.log parser and common fields	**COMPLETED**	BOM handling, nine events, zero unparsed sample lines, sudo parsing	Broader Linux auditd/journald coverage and versioned common schema
Prototype detection engine	**PARTIAL**	Four tested detections working	External YAML engine with 35-45 tested rules and filters
MITRE mapping	**PARTIAL**	Prototype mapping for four rules	Official v19.x catalog sync, sub-techniques, confidence, coverage matrix and gaps
Risk scoring	**PARTIAL**	Deterministic prototype scores generated	Versioned contextual scoring with reason codes and calibration
Alert correlation	**PARTIAL**	Basic source-IP grouping and two incidents	Entity/time/cross-source sequences, suppression and attack graphs
Timeline builder	**COMPLETED**	Events and alerts shown in ordered timeline	Case-specific filtering, graph integration and timezone handling
Streamlit dashboard	**PARTIAL**	Upload, overview, events, alerts, incidents, timeline and report pages	History, case workflow, ATT&CK coverage, RBAC, RAG and API-backed UI
PDF reporting	**PARTIAL**	Working two-page prototype report	Pagination-safe professional templates, executive report and evidence references
Automated core testing and quality	**COMPLETED**	Five tests pass; Ruff and compile checks pass	80%+ coverage, integration, performance and security tests
Git branching and version tags	**COMPLETED**	v0.2.0 and feature/persistent-soc-	Release notes, CI gates and v1.0

Workstream	Current status	Evidence completed	Industrial target
		data branch	release
SQLAlchemy persistent schema	**COMPLETED**	Models for sessions, events, alerts, incidents, notes, evidence and audit logs	PostgreSQL profile, retention and backup strategy
Alembic migration	**COMPLETED**	Initial migration generated and upgraded to head	Subsequent reviewed migrations and upgrade/downgrade tests
Transactional persistence service	**NOT STARTED**	Schema exists only; app does not yet save analyses	Atomic persistence, duplicate protection, history and repositories
Case-management operations and UI	**NOT STARTED**	Database fields exist	Assignment, status, notes, evidence, audit and SLA
Windows/Sysmon/PowerShell connectors	**NOT STARTED**	-	Parser, fixtures, rules and documentation
Suricata/Zeek/firewall connectors	**NOT STARTED**	-	Parser, fixtures, network rules and cross-source correlation
Cloud and identity connectors	**NOT STARTED**	-	CloudTrail and Entra/M365 support
FastAPI backend	**NOT STARTED**	-	Secure API with OpenAPI, RBAC and service layer
ChromaDB, embeddings and RAG	**NOT STARTED**	Architecture selected	Curated knowledge ingestion, retrieval, citations and evaluation
Ollama and Qwen integration	**NOT STARTED**	Model candidate selected	Hardware benchmark, structured prompts and guardrails
Docker Compose deployment	**NOT STARTED**	-	UI, API, PostgreSQL, Chroma, Ollama and health checks

18.1 Verified prototype evidence

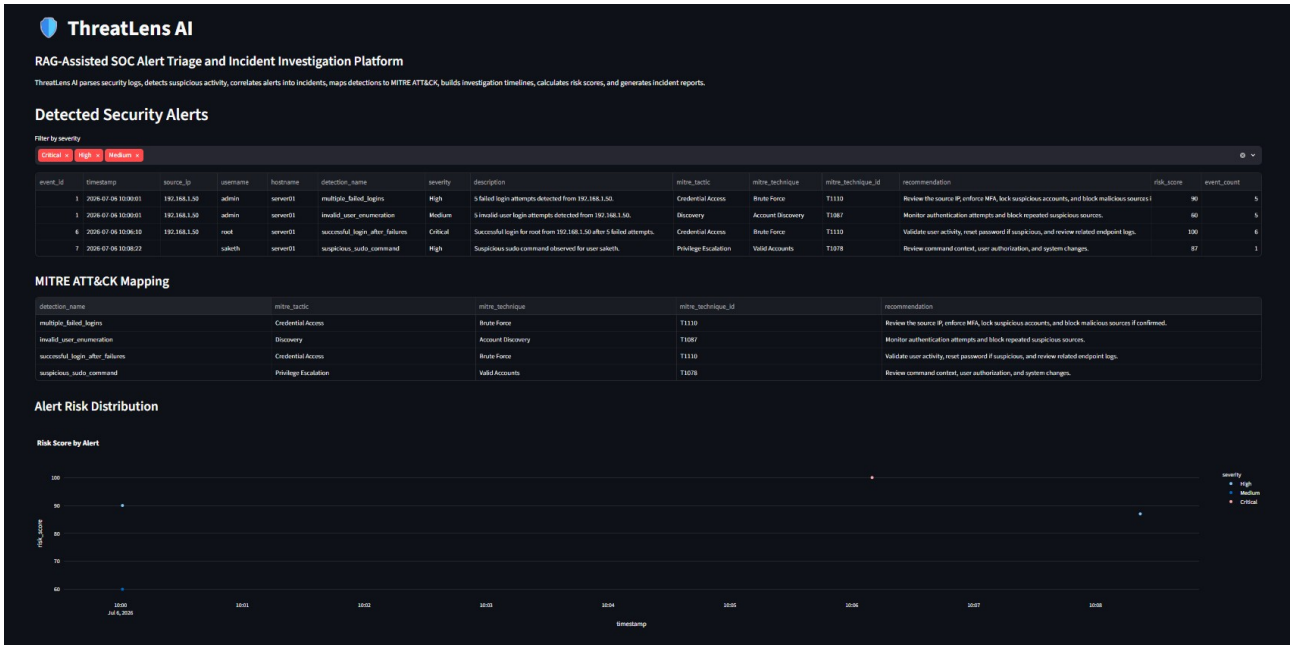


Figure 7. Current alerts, MITRE mappings and risk distribution

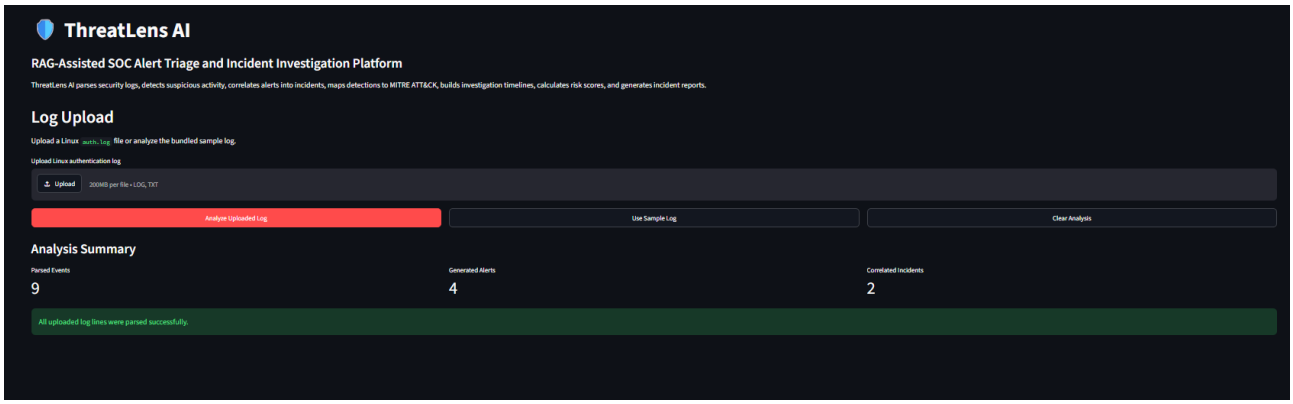


Figure 8. Current log upload and analysis summary

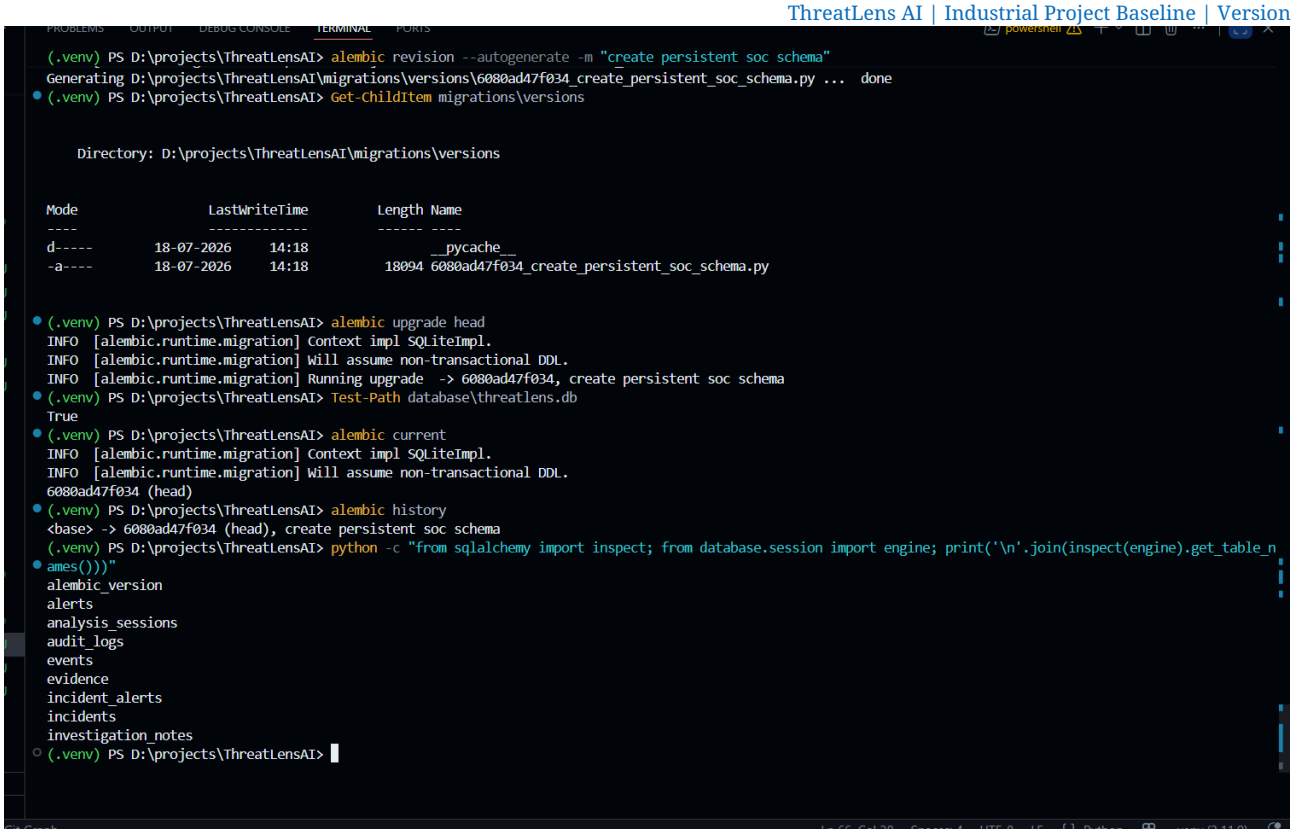


Figure 9. Applied Alembic migration and persistent table verification

19. Master Roadmap and Delivery Plan

19.1 Engineering phases

Phase	Workstream	Status	Deliverable
Phase 0	Project foundation	COMPLETED	Environment, structure, Git, documentation
Phase 1	Core Linux analytics	COMPLETED	Parser, detections, risk, correlation, timeline, dashboard, PDF
Phase 2A	Hardening and tests	COMPLETED	BOM/sudo fixes, tests, packaging, Ruff
Phase 2B.1	Persistent schema	COMPLETED	SQLAlchemy models and Alembic migration
Phase 2B.2	Persistence service	NEXT	Transactions, duplicate control, history, notes, evidence, audit
Phase 2B.3	Case-management UI	PLANNED	Incident queue, assignment, workflow, audit views
Phase 3	External rule engine	PLANNED	YAML/Sigma-style parser, operators, versioning and tests
Phase 4	Windows telemetry	PLANNED	Security, Sysmon and PowerShell connectors and rules
Phase 5	Network telemetry	PLANNED	Suricata, Zeek, firewall connectors and rules
Phase 6	ATT&CK service	PLANNED	STIX/TAXII sync, mapping database, heatmap and Navigator export
Phase 7	Advanced correlation	PLANNED	Entity/time sequences, cross-source incidents and attack graphs
Phase 8	FastAPI and RBAC	PLANNED	API contracts, authentication, roles and service-backed UI
Phase 9	RAG assistant	PLANNED	Knowledge ingestion, Chroma, Ollama/Qwen, citations and evaluation
Phase 10	Deployment and security	PLANNED	PostgreSQL, Docker Compose, health, backup and hardening
Phase 11	Final evaluation	PLANNED	Performance, security, acceptance, reports, PPT, demo and v1.0

19.2 Honest duration estimate

Because the scope has been expanded from a semester prototype to an industrial demonstration, the remaining work should be planned for approximately 14-18 focused weeks, depending on daily effort and hardware. A realistic estimate is 350-500 engineering hours including research, implementation, testing, documentation and presentation. The plan should not be compressed by skipping tests, security or evaluation.

Weeks	Focus
1-2	Persistence service, repositories, duplicate protection and database integration tests
3	Case-management UI, history, audit and evidence
4-5	YAML rule engine, rule validation, operators and test harness
6-8	Windows, Sysmon, PowerShell, Suricata and Zeek connectors/rules
9	ATT&CK synchronization, mapping service and coverage dashboard
10-11	Advanced correlation, suppression, storylines and attack graphs
12	FastAPI, authentication and RBAC
13-14	ChromaDB, knowledge ingestion, Ollama/Qwen and RAG guardrails
15	PostgreSQL, Docker Compose, backup and health checks
16-17	Performance, security, AI and user-acceptance evaluation
18	Final report, PPT, GitHub release, demo recording and interview preparation

20. Limitations, Drawbacks and Risk Register

20.1 Inherent limitations

Limitation/drawback	Consequence	Mitigation
Telemetry dependence	A technique cannot be detected when the required fields or data source are absent.	Display coverage gaps and required telemetry.
False positives	Legitimate administrative or automated behavior may trigger rules.	Filters, allowlists, tuning, confidence and analyst feedback.
False negatives	Novel, obfuscated or low-and-slow behavior may avoid deterministic rules.	Diverse telemetry, behavior correlations and continuous rule improvement.
ATT&CK ambiguity	One behavior may map to multiple techniques and ATT&CK evolves.	Mapping justification, confidence, versioning and review.
RAG hallucination	Retrieval reduces but does not eliminate unsupported generation.	Citations, refusal, evaluation and no autonomous response.
Local LLM resources	Memory and latency may be high on a student laptop.	Benchmark 4B/8B models, quantization and optional API abstraction.
SQLite concurrency	SQLite is limited for concurrent production-style workloads.	Use PostgreSQL in the final deployment profile.
Data sensitivity	Logs contain identities and operational details.	Synthetic datasets, RBAC, redaction, retention and local processing.
Rule maintenance	A large catalog can become stale or inconsistent.	Rule ownership, versions, tests, changelogs and deprecation.
Scope complexity	Many technologies increase integration risk.	Milestone gates, modular boundaries and definition of done.

20.2 Project risk register

ID	Risk	Likelihood	Impact	Mitigation
R1	Scope expansion causes incomplete final product	High	High	Freeze v1.0 scope and require phase acceptance before new features
R2	Parser fields differ across sample datasets	High	Medium	Versioned fixtures, mapping profiles and quarantine errors
R3	Rule engine produces excessive false positives	Medium	High	Negative tests, filters, confidence and tuning dashboard
R4	ATT&CK sync changes schema or objects	Medium	Medium	Pin version, store source version and test synchronization
R5	LLM is too slow on hardware	High	Medium	4B model, quantization, timeouts and fallback templates
R6	RAG answer is not	Medium	High	Evaluation set, citations,

ID	Risk	Likelihood	Impact	Mitigation
	grounded			refusal and source restrictions
R7	Database migration breaks data	Low	High	Backups, integration tests and migration review
R8	Demo fails due to service startup	Medium	High	Health checks, startup script, cached model and offline demo data

21. Demonstration Scenarios

Scenario	Input/story	Expected demonstration
Scenario 1 - Linux account compromise	Five failed SSH attempts, successful root login and sensitive sudo access	Password guessing alert, success-after-failure, privilege alert, correlated incident, ATT&CK timeline and report
Scenario 2 - Windows malicious execution	Encoded PowerShell followed by suspicious process/network behavior	Execution and defense-related mappings, host/user correlation and case workflow
Scenario 3 - Network reconnaissance and C2	Port scan, suspicious DNS and outbound HTTPS from the same host	Network alerts, C2 storyline, graph and containment playbook
Scenario 4 - Cross-source incident	Identity login, endpoint execution and Suricata alert linked by user/host/time	One multi-stage incident instead of separate alerts
Scenario 5 - RAG analyst query	Ask why the incident is P1 and what to investigate next	Cited answer from incident evidence, ATT&CK and approved playbook; refusal for unsupported claims
Scenario 6 - Case audit	Assign analyst, add note, register evidence and resolve case	Persistent workflow and immutable audit history after restart

22. Viva and Interview Question Bank

Q: What is ThreatLens AI?

A: A multi-source security analytics and SOC investigation platform that converts raw telemetry into ATT&CK-aligned, prioritized and auditable incidents, with a grounded RAG assistant.

Q: What real problem does it solve?

A: It reduces the manual effort required to normalize logs, triage repeated alerts, correlate attack stages, preserve case history and explain incidents.

Q: Is it a SIEM?

A: It implements selected SIEM-like analytics and investigation functions, but version 1.0 is an industrial demonstration, not a replacement for a production enterprise SIEM.

Q: Why did you start with Linux auth.log?

A: It provides a controlled source for validating the entire pipeline: parsing, detection, scoring, correlation, timeline, dashboard, report and tests.

Q: Will it support other logs?

A: Yes. The target includes Windows Security/Sysmon/PowerShell, Suricata, Zeek, firewall, cloud and identity telemetry through modular connectors.

Q: Why normalize logs?

A: Rules and correlation cannot operate consistently when every source uses different field names and types. A common schema creates a stable contract.

Q: Why not use only machine learning?

A: Deterministic rules are explainable, testable and controllable. ML can be added later for anomaly support, but it should not replace evidence-based detections.

Q: What is MITRE ATT&CK?

A: A structured knowledge base of adversary tactics and techniques. ThreatLens uses it as a common language for detection coverage and incident explanation.

Q: Can the system map all ATT&CK techniques?

A: It can synchronize and display the complete catalog. It maps an alert only when the rule and available telemetry support that technique.

Q: Why is forced 100% ATT&CK coverage wrong?

A: Different techniques require different telemetry. Claiming coverage without sensors or validated rules is misleading.

Q: How are mappings maintained?

A: Official ATT&CK data is synchronized using versioned STIX/TAXII data, and rule mappings include justification, confidence and required telemetry.

Q: What is a Sigma-style rule?

A: A YAML representation of detection metadata and conditions. The project uses a controlled subset and adds tests, versioning and application-specific operators.

Q: How do you reduce false positives?

A: Negative fixtures, filters, allowlists, thresholds, rule confidence, asset context and analyst feedback.

Q: What is alert correlation?

A: Combining related alerts using entities, time, sequences and behavior to create one incident storyline.

Q: Why are attack graphs useful?

A: They show relationships between users, hosts, IPs, processes, alerts and techniques that are difficult to understand from a flat table.

Q: How is risk calculated?

A: A versioned formula combines severity, confidence, asset criticality, privilege, attack progression, exposure and recurrence, with stored reason codes.

Q: Why use SQLAlchemy?

A: It provides typed ORM models, relationships and transactional data access while allowing the database backend to change.

Q: Why use Alembic?

A: It versions database schema changes and makes upgrades reproducible instead of manually editing tables.

Q: Why SQLite first?

A: It is simple for local development and tests. PostgreSQL is the final industrial demonstration target for concurrency and operational realism.

Q: What does atomic persistence mean?

A: Events, alerts, incidents and relationships are committed together. On failure, the complete transaction is rolled back.

Q: How do you prevent duplicate analyses?

A: SHA-256 content hashes, engine version and database uniqueness/fingerprint checks.

Q: What is RAG?

A: A pattern that retrieves relevant documents before generation so the LLM can answer using approved context.

Q: Why use ChromaDB?

A: It stores documents, embeddings and metadata and supports similarity and filtered retrieval for the knowledge base.

Q: Why use Ollama?

A: It provides a local model runtime and API, reducing external data transmission and allowing controlled offline demonstrations.

Q: Why Qwen3 4B?

A: It is a practical starting point for local hardware. The final choice will be based on benchmarked groundedness, latency and memory.

Q: Does RAG eliminate hallucination?

A: No. It reduces risk but still requires source controls, citations, evaluation and refusal behavior.

Q: Can the AI detect attacks?

A: No. The deterministic analytics engine detects and scores. AI explains and assists after evidence exists.

Q: How is prompt injection handled?

A: Approved sources, separation of instructions and data, metadata filtering, input sanitization and output validation.

Q: How is evidence integrity protected?

A: Evidence records can store SHA-256 hashes, references, timestamps and audit actions.

Q: How is the project tested?

A: Unit, integration, pipeline, API, UI, performance, security, RAG and user-acceptance tests.

Q: What is completed now?

A: Core Linux pipeline, dashboard, prototype PDF, tests, Git versioning, SQLAlchemy schema and Alembic migration.

Q: What is the next feature?

A: Transactional persistence service followed by case-management history and UI.

Q: What are the biggest drawbacks?

A: Telemetry dependence, false positives/negatives, maintenance, local LLM constraints, privacy and scope complexity.

Q: How will it be deployed?

A: A Docker Compose demonstration with Streamlit, FastAPI, PostgreSQL, ChromaDB, Ollama and persistent volumes.

Q: What makes this project industry-level?

A: Service boundaries, persistence, migrations, rule lifecycle, ATT&CK sync, case management, audit, RAG guardrails, tests, deployment and measurable acceptance criteria.

23. Conclusion and Baseline Decision

ThreatLens AI will continue as an advanced industrial demonstration project. The project will not be reduced to a Linux log dashboard or a generic AI chatbot. The completed core proves the end-to-end concept, while the persistent schema establishes the foundation for history, case management and auditability.

The immediate next task is the transactional persistence service and its integration tests. After persistence and case-management UI are complete, the project will progress to an external rule engine, multi-source telemetry, official ATT&CK synchronization, advanced correlation, FastAPI/RBAC, RAG, containerized deployment and final evaluation.

Baseline commitment

No feature will be marked complete without working output, automated tests, documentation and acceptance evidence. The final release will make honest claims: complete catalog awareness, measured detection coverage, explicit telemetry gaps and grounded AI assistance.

24. References

Ref.	Source
[1]	MITRE ATT&CK, Updates - April 2026 and Enterprise tactics/techniques pages. Current ATT&CK v19 statistics and 15 Enterprise tactics. Retrieved 18 July 2026.
[2]	SigmaHQ, Sigma Rules documentation. YAML detection-rule structure and purpose. Retrieved 18 July 2026.
[3]	SigmaHQ, Correlations documentation. Event-count, temporal, ordered and group-by correlation concepts. Retrieved 18 July 2026.
[4]	MITRE ATT&CK, ATT&CK Data and Tools / Working with ATT&CK. Official STIX and TAXII access. Retrieved 18 July 2026.
[5]	Chroma documentation, Introduction and Manage Collections. Collections, documents, metadata and embeddings. Retrieved 18 July 2026.
[6]	Chroma documentation, Query and Get. Dense similarity retrieval and metadata/document filtering. Retrieved 18 July 2026.
[7]	Ollama documentation and Qwen3 model library. Local API and available Qwen3 model sizes. Retrieved 18 July 2026.
[8]	SQLAlchemy documentation, Session Basics and Transaction Management. Commit, rollback and transactional sessions. Retrieved 18 July 2026.
[9]	Alembic documentation. SQLAlchemy database migrations and autogeneration. Retrieved 18 July 2026.
[10]	FastAPI official documentation, Features. OpenAPI and automatic interactive API documentation. Retrieved 18 July 2026.
[11]	Docker documentation, Docker Compose. Definition and operation of multi-container applications. Retrieved 18 July 2026.

Appendix A - Target Repository Structure

```
ThreatLensAI/
|-- app/                 # Streamlit presentation layer
|-- api/                 # FastAPI routes, schemas and dependencies
|-- core/                # parsers, detection, risk, correlation, timeline
|-- domain/              # common event, alert and incident models
|-- rules/               # YAML rules, filters and correlation definitions
|   |-- linux/
|   |-- windows/
|   |-- network/
|   `-- cloud/
|-- attack/              # ATT&CK sync, mapping and Navigator export
|-- database/            # SQLAlchemy models, repositories and services
|-- migrations/          # Alembic migration history
|-- rag/                 # ingestion, retrieval, prompts and evaluation
|-- knowledge_base/      # approved ATT&CK/playbook/rule documents
|-- reports/             # templates and rendering logic
|-- config/              # versioned non-secret configuration
|-- tests/               # unit, integration, pipeline and security tests
|-- data/sample_logs/    # synthetic authorized fixtures
|-- docker/              # container files and health checks
|-- docs/                # architecture, ADRs, operations and project report
|-- compose.yaml
|-- pyproject.toml
`-- README.md
```

Appendix B - Current Database Tables

Table	Purpose
analysis_sessions	Metadata and counts for one ingestion/analysis run
events	Normalized security events and fingerprints
alerts	Rule findings, risk, confidence and ATT&CK mappings
incidents	Correlated investigation cases and workflow state
incident_alerts	Many-to-many relationship between incidents and alerts
investigation_notes	Analyst notes
evidence	Evidence metadata, references and integrity hash
audit_logs	Security-sensitive action history
alembic_version	Current migration revision

Appendix C - Immediate Next Implementation Gate

The next phase is complete only when all of the following are true:

- A complete analysis is persisted in one transaction.
- Duplicate content is rejected unless explicitly overridden.
- Events, alerts, incidents and incident-alert relationships can be reloaded.
- Analysis history remains after Streamlit restart.
- Incident status, priority and assignee can be updated.
- Notes and evidence can be added and are visible in case details.
- Audit records are created for persistence and workflow changes.
- Integration tests use an isolated database and pass.
- Ruff, compilation, Alembic check and the existing core tests pass.

Appendix D - Resume Description After Version 1.0

ThreatLens AI - RAG-Assisted Multi-Source SOC Investigation Platform | Python, FastAPI, Streamlit, PostgreSQL, SQLAlchemy, Alembic, MITRE ATT&CK, Sigma-style YAML, ChromaDB, Ollama/Qwen, Docker

- Engineered a multi-source security analytics platform that normalizes Linux, Windows, Sysmon, Suricata, Zeek and cloud/identity telemetry into a common event schema.
- Implemented versioned detection rules, contextual risk scoring and entity/time-based alert correlation to create ATT&CK-aligned incident storylines and attack graphs.
- Built persistent SOC case management with analyst assignment, evidence integrity hashes, investigation notes, immutable audit logs and migration-controlled storage.
- Integrated a grounded RAG analyst assistant using ChromaDB and a local Ollama-hosted Qwen model to produce cited explanations and response guidance without replacing deterministic detections.
- Containerized the UI, API, PostgreSQL, vector database and LLM services with automated tests, health checks, professional reports and ATT&CK Navigator exports.